Memory Layout, GPIO, Register und RCC

Andreas Scheibenpflug MSc.
andreas.scheibenpflug@fh-hagenberg.at

Bachelor-Studiengang
Software Engineering

Fakultät für Informatik, Kommunikation und Medien

Hagenberg, FH-OÖ

1. Memory
   - Einführung
   - Memory Layout
2. GPIO
   - Allgemeines
   - Konfiguration
   - Exkurs
   - Konfiguration
3. Exkurs: Register und Bitweise Operationen in C
4. Reset and Clock Control
   - Überblick
   - Konfiguration
5. Zusammenfassung
   - Beispiel

Inhalt

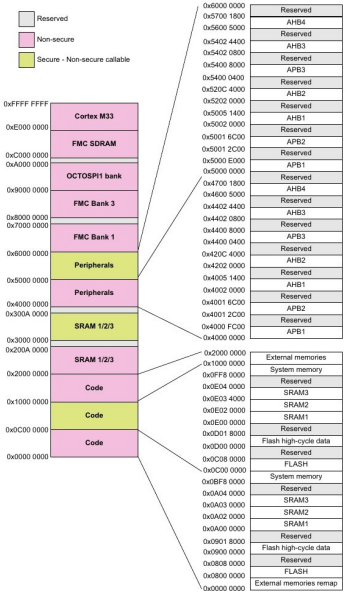
1. Memory
   - Einführung
   - Memory Layout
2. GPIO
3. Exkurs: Register und Bitweise Operationen in C
4. Reset and Clock Control
5. Zusammenfassung

Überblick

- Der Speicher eines Mikrocontrollers ist in verschiedene Bereiche unterteilt
- Jeder Bereich hat eine spezifische Funktion
  - Flash-Speicher
  - SRAM (Static random-access memory)
  - Peripherie-Register
- Flash Speicher ist nicht flüchtig und enthält den kompilierten Programmcode
- SRAM ist flüchtig und wird für Text (Code), Variablen und Stack zur Laufzeit des Programms verwendet
- Peripherie-Register sind spezielle Speicheradressen, die zur Steuerung von Hardware-Komponenten verwendet werden
- Alle drei Speicherbereiche befinden sich im selben (32-bit, in unserem Fall) Adressraum

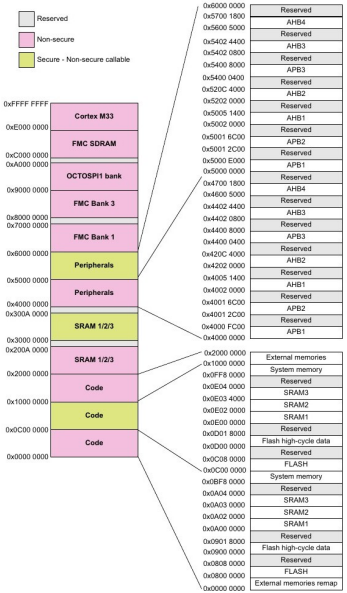
Memory Layout (1/2)

- Auch Memory Map genannt
- Peripherals: Adressbereich der Busse und daran angeschlossener Peripherie
- SRAM: Adressbereich für statischen RAM
- Code: Adressbereich für FLASH Speicher
- FLASH Speicher Blockgröße 8Kb
- Gelbe Bereiche sind TrustZone Erweiterungen (werden wir nicht verwenden)
- Siehe Reference Manual, Seite 117



Memory Layout (2/2)

- AHB und APB sind unterschiedliche Bussysteme
- AHB2 und 4 sind für Peripherie
- AHB1 und 3 sind zusätzlich APB Bridges
- F(S)MC (Flexible Memory Controller) → Einbinden von externem Speicher in den Addressraum
- OCTOSPI → High Speed External Memory Interface



Zusammenhang Linker Script

- Das Linker Script definiert u.A. die Speicherbereiche für Code und Daten
- Es legt fest, wo im Speicher die verschiedenen Abschnitte platziert werden
- Beispiel: Der .text Abschnitt (Code) wird im FLASH Speicher platziert
- Der .data Abschnitt (initialisierte Variablen) wird im SRAM platziert

```
MEMORY /* Memories definition */
{
RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH = 272K
FLASH    (rx)    : ORIGIN = 0x08000000,   LENGTH = 512K
}
...
SECTIONS /* Layout of address space based on order and above definitions */
{
...
.text : /* The program code and other data into "FLASH" Rom type memory */
{
...
} >FLASH
...
.data : /* Initialized data sections into "RAM" Ram type memory */
{
...
} >RAM AT> FLASH
...
}
```

Memory Layout - Bus

- Beispiel Bus AHB2 (Reference Manual Seite 124)
- Bus AHB2 ist für die Anbindung von Peripheriekomponenten zuständig
- AHB ist schneller als APB, hat dafür aber höheren Stromverbrauch
- Meisten Busse müssen vor Benutzung aktiviert werden ($\rightarrow$ Stromverbrauch)

Bus	Secure boundary address	Non-secure boundary address	Peripheral	Register map
AHB2	0x520C 4000 - 0x53FF FFFF	0x420C 4000 - 0x43FF FFFF	Reserved	-
	0x520C 2000 - 0x520C 3FFF	0x420C 2000 - 0x420C 3FFF	PKA + RAM	PKA
	0x520C 1000 - 0x520C 1FFF	0x420C 1000 - 0x420C 1FFF	Reserved	-
	0x520C 0C00 - 0x520C 0FFF	0x420C 0C00 - 0x420C 0FFF	SAES	SAES
	0x520C 0800 - 0x520C 0BFF	0x420C 0800 - 0x420C 0BFF	RNG	RNG
	0x520C 0400 - 0x520C 07FF	0x420C 0400 - 0x420C 07FF	HASH	HASH
	0x520C 0000 - 0x520C 03FF	0x420C 0000 - 0x420C 03FF	AES	AES
	0x5202 C800 - 0x520B FFFF	0x4202 C800 - 0x420B FFFF	Reserved	-
	0x5202 C400 - 0x5202 C7FF	0x4202 C400 - 0x4202 C7FF	PSSI	PSSI
	0x5202 C000 - 0x5202 C3FF	0x4202 C000 - 0x4202 C3FF	DCMI	DCMI
	0x5202 8800 - 0x5202 BFFF	0x4202 8800 - 0x4202 BFFF	Reserved	-
	0x5202 8400 - 0x5202 87FF	0x4202 8400 - 0x4202 87FF	DAC1	DAC
	0x5202 8000 - 0x5202 83FF	0x4202 8000 - 0x4202 83FF	ADC1 / ADC2	ADC
	0x5202 2000 - 0x5202 7FFF	0x4202 2400 - 0x4202 7FFF	Reserved	-
	0x5202 1C00 - 0x5202 1FFF	0x4202 1C00 - 0x4202 1FFF	GPIOH	GPIO
	0x5202 1800 - 0x5202 1BFF	0x4202 1800 - 0x4202 1BFF	GPIOG	
	0x5202 1400 - 0x5202 17FF	0x4202 1400 - 0x4202 17FF	GPIOF	
	0x5202 1000 - 0x5202 13FF	0x4202 1000 - 0x4202 13FF	GPIOE	
	0x5202 0C00 - 0x5202 0FFF	0x4202 0C00 - 0x4202 0FFF	GPIOD	
	0x5202 0800 - 0x5202 0BFF	0x4202 0800 - 0x4202 0BFF	GPIOC	
	0x5202 0400 - 0x5202 07FF	0x4202 0400 - 0x4202 07FF	GPIOB	
	0x5202 0000 - 0x5202 03FF	0x4202 0000 - 0x4202 03FF	GPIOA	

Inhalt

1 Memory

2 GPIO
- Allgemeines
- Konfiguration
- Exkurs
- Konfiguration

3 Exkurs: Register und Bitweise Operationen in C

4 Reset and Clock Control

5 Zusammenfassung

Einleitung

- Mikrocontroller können verschiedenste Peripherie verwenden und ansteuern
- Oft Hauptaufgabe eines eingebetteten Systems. Es sollen
  - Aktoren (Motoren, Displays, LEDs,...) angesteuert und/oder
  - Sensoren (Temperatur, Licht, Schalter,...) abgefragt werden
- Sensoren liefern oft den Zustand/Status eines Steuerungsvorgangs
- GPIOs ermöglichen die Ansteuerung und Abfrage von digitalen Geräten
- Sie können als digitale Ein- oder Ausgänge verwendet werden
- GPIOs können interne oder nach Außen geführte PINs sein

GPIO - General Purpose Input/Output

- GPIO Pins sind universelle Ein- und Ausgänge
- Können als digitale Ein- oder Ausgänge konfiguriert werden
- Jeder GPIO Pin kann individuell konfiguriert werden
- GPIO Ports bestehen aus mehreren Pins (max. 16 Pins, z.B. Port A hat 16 Pins: PA0 bis PA15)
- H533RE besitzt 8 GPIO Ports, welche über den Bus AHB2 erreichbar sind
- Die Ports werden mit Buchstaben durchnummeriert: GPIOA - GPIOH
- Nach einem Reset sind alle Ports als Input konfiguriert
- $\rightarrow$ angeschlossene Peripherie kann sich nicht selbständig machen oder zerstören

GPIO - Pin Konfiguration

- Vier Betriebsarten:
  - Input (z.B. Taster)
  - Output (z.B. LED)
  - Alternate Function (z.B. UART)
  - Analog (für DAC/ADC)
- Konfigurationsmöglichkeiten (Auszug):
  - Pull-Up/Pull-Down Widerstand (None, Pull-Up, Pull-Down)
  - Output Typ (Push-Pull oder Open-Drain)
  - Input Typ (Pull-Up, Pull-Down oder beides)
  - Output Geschwindigkeit, ...
- Konfiguration erfolgt über spezielle Register im GPIO Port
- Zuvor muss noch RCC (Reset and Clock Control) für den entsprechenden Bus/Port aktiviert werden (siehe später)

Exkurs: Logikpegel

- Logikwerte werden durch elektrische Spannung dargestellt
- Binär: Hohe Spannung (H(igh)-Pegel) und niedrige Spannung (L(ow)-Pegel)
- Diesen beiden Pegeln werden die Binärwerte 0 und 1 zugewiesen
- Wenn 0 durch den Low-Pegel dargestellt wird, spricht man von positiver Logik
- Wenn 0 durch den High-Pegel dargestellt wird, spricht man von negativer Logik (z.B. RS-232)
- In unserem Fall sind wir bei 3.3 Volt Logikpegel, was ca. folgenden Bereichen entspricht:
  - Low: 0V bis ca. 0.8V
  - High: ca. 2.0V bis 3.3V
- Zwischen den L- und H Bereichen liegt ein undefinierter Bereich, welcher als Puffer dient

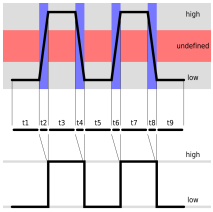


Abbildung: Digitales Signal (Quelle: Wikipedia)

Exkurs: Logikpegel - Beispiel

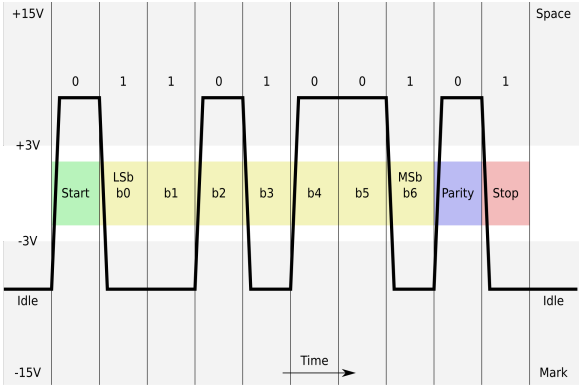


Abbildung: Logikpegel RS-232, Buchstabe 'K' (Quelle: Wikipedia)

Logikpegel und GPIO

- Pull-Up/Down Widerstände ermöglichen einen definierten Logikpegel
- Pull-Up Widerstand zieht den Pegel auf High, Pull-Down Widerstand auf Low
- Push-Pull ist die Standardeinstellung für Ausgänge
  - Wenn Ausgang aktiv ist, wird der Pegel auf High gezogen
  - Wenn Ausgang inaktiv ist, wird der Pegel auf Low gezogen
- Eingänge 'floaten' (sind undefiniert), wenn nicht verwendet
- Pull-ups/downs helfen, den Zustand eines Eingangs zu definieren, wenn kein Signal anliegt
- Wenn ein Signal anliegt, wird der Zustand des Eingangs durch das Signal bestimmt
  - z.B. Taster gegen Masse (Low-Pegel) mit Pull-Up Widerstand
  - Wenn Taster gedrückt wird, wird der Pegel auf Low gezogen
  - Wenn Taster nicht gedrückt wird, zieht der Pull-Up Widerstand den Pegel auf High

GPIO - Konfiguration als Eingang

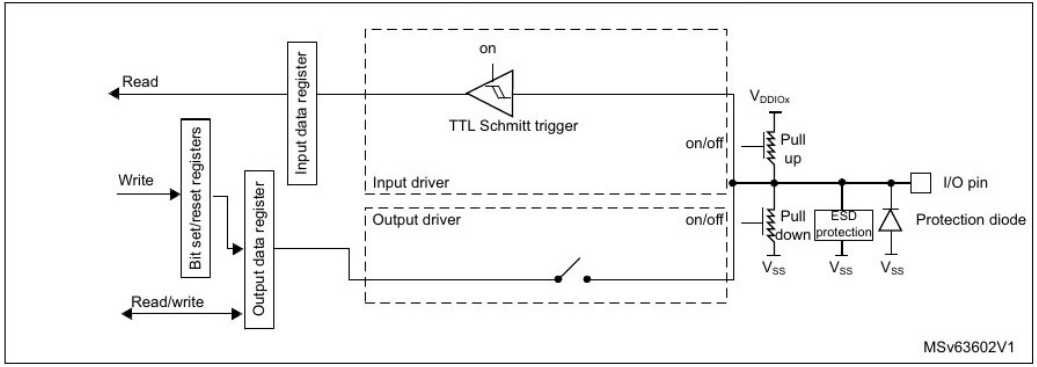


Abbildung: GPIO Konfiguration als Eingang (Quelle: Reference Manual, Seite 580)

GPIO Register

- GPIO Register sind spezielle Speicheradressen, die zur Steuerung der GPIO Ports verwendet werden
- Jedes Register hat eine spezifische Funktion (Auswahl):
  - MODER: Modus Register (Input, Output, Alternate Function, Analog)
  - OTYPER: Output Typ Register (Push-Pull, Open-Drain)
  - PUPDR: Pull-Up/Pull-Down Register (None, Pull-Up, Pull-Down)
  - IDR: Input Data Register (liest den Zustand der Pins)
  - ODR: Output Data Register (setzt den Zustand der Pins)
  - BRR: Bit Reset Register (setzt einzelne Pins zurück)
  - BSRR: Bit Set/Reset Register (setzt oder löscht einzelne Pins)
- Jedes Register ist 32-bit breit und jeder Pin wird durch ein oder zwei Bits in den meisten Registern konfiguriert

MODER

- MODER Register konfiguriert den Modus jedes Pins
- Jeder Pin wird durch zwei Bits im MODER Register konfiguriert
- Mögliche Modi:
  - 00: Input
  - 01: Output
  - 10: Alternate function
  - 11: Analog mode

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Abbildung: MODER Register, Reference Manual, Seite 586

PUPDR

- PUPDR Register konfiguriert die Pull-Up/Pull-Down Widerstände jedes Pins
- Jeder Pin wird durch zwei Bits im PUPDR Register konfiguriert
- Mögliche Modi:
  - 00: No Pull-Up/Down
  - 01: Pull-Up
  - 10: Pull-Down
  - 11: Reserved

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Abbildung: PUPDR Register, Reference Manual, Seite 587

IDR

- IDR Register enthält den Zustand der Pins
- Jeder Pin wird durch ein Bit im IDR Register repräsentiert
- Wenn Pin auf High, ist entsprechendes Bit im IDR Register gesetzt
- Wenn Pin auf Low, ist entsprechendes Bit im IDR Register nicht gesetzt

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Abbildung: IDR Register, Reference Manual, Seite 588

Lage und Adressen

- GPIO Register befinden sich im Adressraum des AHB2 Busses
- Jeder GPIO Port hat eine eigene Basisadresse
- Beispiel: GPIOC Basisadresse ist 0x4202 0800
- MODER Register Offset ist 0x00
- Adresse des MODER Registers von GPIOC ist also 0x4202 0800 + 0x00 = 0x4202 0800
- Adresse des IDR Registers von GPIOC ist 0x4202 0800 + 0x10 = 0x4202 0810

Verwendung mit CMSIS

Konfiguration MODER: Versetzen GPIOC Pin 13 in analog mode

```
GPIOC->MODER |= (3 << (13 * 2)); // Pin 13 in analog mode
```

Verwenden des IDR Registers: Auslesen des Zustands von GPIOC Pin 13

```
uint32_t pinValue = (GPIOC->IDR & (1 << 13)) >> 13; // Pin 13 auslesen
```

Aufbau CMSIS

Beispiel Struktur GPIO_TypeDef (aus stm32h533xx.h):

```
typedef struct
    {
        __IO uint32_t MODER;  /* GPIO port mode register, offset: 0x00 */
        __IO uint32_t OTYPER; /* GPIO port output type register, offset: 0
            x04 */
        ...
        __IO uint32_t PUPDR;  /* GPIO port pull-up/pull-down register,
            offset: 0x0C */
        ...
    } GPIO_TypeDef;
```

Struktur definiert Memory Layout der einzelnen Register eines GPIO Ports.
Mit Hilfe von #defines werden für alle Ports die Basisadressen definiert (vereinfachte Darstellung):

```
#define GPIOC ((GPIO_TypeDef *) (0x42020800UL))
```

Inhalt

1 Memory

2 GPIO

3 Exkurs: Register und Bitweise Operationen in C

4 Reset and Clock Control

5 Zusammenfassung

Exkurs: Register und Bitweise Operationen in C

- Register sind die API zur Hardware
- Register sind spezielle Speicheradressen, die zur Steuerung von Hardware verwendet werden
- Bitweise Operationen sind notwendig, um einzelne Bits in einem Register zu setzen, zu löschen oder zu überprüfen

Beispiel: Setzen eines Bits

```
REG |= (1 << 5); // Setzt Bit an Position 5
```

Beispiel: Löschen eines Bits

```
REG &= ~(1 << 5); // Loescht Bit an Position 5
```

Beispiel: Überprüfen eines Bits

```
if (REG & (1 << 5)) {...} // Check ob Bit an Position 5 gesetzt ist
```

Bitweise Operationen in C - Weitere Beispiele

Beispiel: Setzen eines Bits

```
REG |= 0x20; // Setzt Bit an Position 5
REG |= 0b00100000;
```

Beispiel: Überprüfen eines Bits, erweitert

```
if (REG & 0x20) {
    REG &= ~0x20;
} else {
    REG |= 0x20;
}
```

Beispiel: Bitshift

```
uint32_t REG = 1;
for(int i = 0; i < 8; i++) {
    REG <<= 1;
}
```

Inhalt

1 Memory

2 GPIO

3 Exkurs: Register und Bitweise Operationen in C

4 Reset and Clock Control
- Überblick
- Konfiguration

5 Zusammenfassung

RCC - Allgemeines

- Um GPIOs verwenden zu können, fehlt uns noch ein Schritt zuvor
- Aktivierung/Deaktivierung von Takt für Peripheriekomponenten (GPIO, Timer, ADC/DAC,...)
- RCC ist unter anderem zuständig für die Taktversorgung der Peripherie und die Taktsteuerung der Busse
- Um Strom zu sparen, sind standardmäßig die meisten Busse und Peripheriekomponenten deaktiviert
- Zur Aktivierung muss der Takt mithilfe von RCC gesetzt werden
- Immer dann, wenn ein Reset des Systems erfolgt, z.B.:
  - Power Reset (Power-On/Off, Brown-Out, Standby/Wake)
  - System Reset (z.B. durch Software, Reset Pin,...)
- $\rightarrow$ Es werden alle/oder ein Teil der Register (je nach Reset Typ) auf ihre Reset-Werte zurückgesetzt
- Und damit auch die meisten Busse/Peripherie deaktiviert

RCC - Konfiguration

- Zur Konfiguration des RCC steht eine Auswahl an Clocks zur Verfügung
- Diese unterscheiden sich in Geschwindigkeit, Genauigkeit und Stromverbrauch
- Mögliche Clock Quellen (Auszug):
  - HSI (High-Speed Internal) - interner Oszillator, bis zu 64MHz
  - CSI (Clock Source Internal) - interner Oszillator
  - HSE (High-Speed External) - externer Crystal/Clock, 4-50Mhz
- Die Konfiguration wird normalerweise im System-Startup-Code durchgeführt
  - Auswahl der Clock Quelle
  - Konfiguration der Clock Frequenz
  - Anpassung der Frequenz mit Prescalern (Reduktion der Frequenz) oder Frequency Multiplication (Erhöhung der Frequenz)
- Wir werden uns mit diesem Thema aber nicht im Detail beschäftigen
- plib Funktion `pl_init()` führt die Konfiguration aus

RCC - Aktivieren von Komponenten

- Peripherie ist an unterschiedlichen Bussen angeschlossen
- Es gibt daher für jeden Bus ein eigenes RCC Enable Register
- Erst nach Aktivierung der Clock kann die Peripherie konfiguriert und verwendet werden
- Beispiel Aktivierung der Clock für GPIOC mit CMSIS

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN; // GPIOC aktivieren
```

- `RCC_AHB2ENR_GPIOCEN` ist eine vordefinierte Konstante in der CMSIS Header Datei
- Diese repräsentiert das Bit im Register, welches für die Aktivierung von GPIOC zuständig ist
- Ein Bus kann mit einem Reset Register auch wieder deaktiviert werden

```
RCC->AHB2RSTR |= RCC_AHB2RSTR_GPIOCRST; // GPIOC deaktivieren
```

Inhalt

1 Memory

2 GPIO

3 Exkurs: Register und Bitweise Operationen in C

4 Reset and Clock Control

5 Zusammenfassung
- Beispiel

Beispiel: Konfiguration des User Buttons

Siehe plib Funktion `userbutton_init()`
AHB2 Bus aktivieren

```
SET_BIT(RCC->AHB2ENR, RCC_AHB2ENR_GPIOCEN);
```

Pull-up/Pull-down Register auf Pull-Down setzen. 0 ist damit der Default(Reset)-Wert, wenn der Taster nicht gedrückt ist

```
MODIFY_REG(GPIOC->PUPDR, GPIO_PUPDR_PUPD13_Msk, GPIO_PULLDOWN <<
    GPIO_PUPDR_PUPD13_Pos);
```

Pin 13 (User Button) mit Hilfe des MODER Registers auf Input setzen

```
MODIFY_REG(GPIOC->MODER, GPIO_MODER_MODE13_Msk, GPIO_MODE_INPUT <<
    GPIO_MODER_MODE13_Pos);
```

Pin 13 mit Hilfe des IDR Registers auslesen

```
uint32_t pinValue = GPIOC->IDR & (1 << 13);
```